

COMPILER CONSTRUCTION PROJECT

Project Report:
LEX and YACC Compiler

Project Title: Mini Compiler using LEX & YACC

Group ID : G1-04

Student IDs:

- **445007595**
- **445005974**
- **445004855**
- **445006007**
- **445006891**

Students Names :

نهله منسي البقمي
ضحى ناصر النباتي
سوسن محمد عمر
تهاني عامر البارقي
عائشة دخیل الله العدواني

1- INTRODUCTION:

This project aims to simulate a simplified compiler using LEX and YACC. It is divided into two phases:

- Lexical Analysis using LEX: To identify tokens such as keywords, identifiers, numbers, and symbols from the source code.
- Syntax Analysis using YACC: To process the structure of valid statements and expressions based on defined grammar rules.

LEX and YACC work together to convert raw source code into a structured format suitable for further compilation or interpretation.

2- PART 1: LEXICAL ANALYSIS

TASK I: RESEARCH LEX AND ANSWER THE FOLLOWING QUESTIONS.

Q1: What is LEX?

LEX is a program that helps build compilers. It works in the code analysis phase and breaks the code into small parts called "tokens" like keywords, variable names, numbers, and symbols. These tokens help the computer understand the code and continue processing it in later stages.

Q2: How does LEX work?

LEX reads text letter by letter, matches it with specific patterns using regular expressions, and performs an appropriate action when finding a match, such as returning tokens. LEX always prefers the longest possible match first.

Q3: What is the syntax of LEX code?

LEX program consists of three main sections:

Definitions Section:

This section is used to define macros, regular expressions, and include any necessary header files.

Rules Section:

It contains a series of pattern-action pairs.

User Code Section:

you can include any additional C code, such as the main() function or other helper functions, that are needed to support the lexer.

Each section is separated by the delimiter %%.

Q4: Can you use an IDE for LEX development?

Yes, I can use IDE to develop LEX code . Good choices include VS CODE

LEX CODE:

Note:
This code is before being linked to the parser code.

```
Lex.l
1  %{
2  #include <stdio.h>
3  #include <string.h>
4  #include <stdlib.h>
5
6  #define SYMBOLS_SIZE 500
7  char* symbol_table[SYMBOLS_SIZE];
8  int symbol_count = 0;
9
10 int yywrap(void) { return 1; }
11
12 void insert_symbol(const char* lexeme) {
13     for (int i = 0; i < symbol_count; ++i) {
14         if (strcmp(symbol_table[i], lexeme) == 0)
15             return;
16     }
17     symbol_table[symbol_count++] = strdup(lexeme);
18 }
19
20 void print_symbol_table() {
21     printf("\n--- Symbol Table ---\n");
22     for (int i = 0; i < symbol_count; ++i) {
23         printf("%s\n", symbol_table[i]);
24     }
25 }
26
27 #define ASSIGN 1
28 #define LESSEQUAL 2
29 #define GREATEREQUAL 3
30 #define NOTEQUAL 4
31 #define DOTDOT 5
32 #define PLUS 6
33 #define MINUS 7
34 #define MULT 8
35 #define EQUAL 9
36 #define LESS 10
37 #define GREATER 11
38 #define COLON 12
39 #define SEMICOLON 13
40 #define COMMA 14
41 #define LPAREN 15
42 #define RPAREN 16
43 #define LBRACKET 17
44 #define RBRACKET 18
```

LEX CODE:

```
Lex.l
/*
71  #define ID 100
72  #define INT 101
73  #define STRING 102
74  %}
75
76  DIGIT    [0-9]
77  LETTER   [a-zA-Z]
78  ID       {LETTER}{(LETTER){DIGIT}|_)*
79  INT      {DIGIT}+
80  STR      \"[^\"]*\"
81  COMMENT  \"{\"[^\"]*\"}\"
82
83  %%
84
85  [ \t\r\n]+    { /* ignore all whitespaces including newline and carriage return */ }
86
87  {COMMENT}     { /* ignore comments */ }
88
89  "program"     { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return PROGRAM; }
90  "var"         { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return VAR; }
91  "begin"       { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return BEGIN_KEYWORD; }
92  "end"         { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return END; }
93  "if"          { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return IF; }
94  "then"        { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return THEN; }
95  "else"        { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return ELSE; }
96  "and"         { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return AND; }
97  "do"          { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return DO; }
98  "for"         { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return FOR; }
99  "div"         { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return DIV; }
100 "forward"     { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return FORWARD; }
101 "function"    { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return FUNCTION; }
102 "array"       { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return ARRAY; }
103 "mod"         { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return MOD; }
104 "not"         { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return NOT; }
105 "of"          { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return OF; }
106 "or"          { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return OR; }
107 "procedure"   { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return PROCEDURE; }
108 "record"      { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return RECORD; }
109 "to"          { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return TO; }
110 "type"        { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return TYPE; }
111 "while"       { insert_symbol(yytext); printf("KEYWORD: %s\n", yytext); return WHILE; }
112
113 \"[^\"]*\"     { printf("STRING: %s\n", yytext); return STRING; }
```

LEX CODE:

```
Lex.l
113 \("[^"])*"      { printf("STRING: %s\n", yytext); return STRING; }
114 [0-9]+         { printf("INT: %s\n", yytext); return INT; }
115
116 [a-zA-Z_][a-zA-Z0-9_]* {
117     insert_symbol(yytext);
118     printf("ID: %s\n", yytext);
119     return ID;
120 }
121
122 "=="           { printf("SYMBOL: ==\n"); return ASSIGN; }
123 "<="           { printf("SYMBOL: <=\n"); return LESSEQUAL; }
124 ">="           { printf("SYMBOL: >=\n"); return GREATEREQUAL; }
125 "<>"          { printf("SYMBOL: <>\n"); return NOTEQUAL; }
126 ".."           { printf("SYMBOL: ..\n"); return DOTDOT; }
127 "+"            { printf("SYMBOL: +\n"); return PLUS; }
128 "-"            { printf("SYMBOL: -\n"); return MINUS; }
129 "*"            { printf("SYMBOL: *\n"); return MULT; }
130 "="            { printf("SYMBOL: =\n"); return EQUAL; }
131 "<"            { printf("SYMBOL: <\n"); return LESS; }
132 ">"            { printf("SYMBOL: >\n"); return GREATER; }
133 ":"            { printf("SYMBOL: :\n"); return COLON; }
134 ";"            { printf("SYMBOL: ;\n"); return SEMICOLON; }
135 ","            { printf("SYMBOL: ,\n"); return COMMA; }
136 "("            { printf("SYMBOL: (\n"); return LPAREN; }
137 ")"            { printf("SYMBOL: )\n"); return RPAREN; }
138 "["            { printf("SYMBOL: [\n"); return LBRACKET; }
139 "]"            { printf("SYMBOL: ]\n"); return RBRACKET; }
140 "."            { printf("SYMBOL: .\n"); return DOT; }
141
142 .              { printf("UNKNOWN: %s\n", yytext); }
143
144 %%
145
146 int main() {
147     while(yylex());
148     print_symbol_table();
149     return 0;
150 }
151
```

CODE EXPLANATION:

THIS CODE IS A LEXICAL ANALYZER (LEXER), DESIGNED TO PROCESS A PASCAL-LIKE PROGRAMMING LANGUAGE. IT ANALYZES SOURCE CODE AND BREAKS IT INTO TOKENS SUCH AS KEYWORDS (PROGRAM, IF, VAR), IDENTIFIERS, NUMBERS, STRINGS, AND OPERATORS WHILE IGNORING WHITESPACE AND COMMENTS. IT MAINTAINS A SYMBOL TABLE TO TRACK DECLARED VARIABLES AND PREVENT DUPLICATES. THE LEXER WORKS WITH A SUBSEQUENT PARSER TO VALIDATE SYNTAX, RETURNING EACH TOKEN WITH ITS ASSOCIATED VALUE (E.G., VARIABLE NAMES OR NUMBERS) VIA YYLVAL.

HOW TO RUN THE CODE?!

TO TEST THE LEXICAL ANALYZER ONLY, WE FOLLOWED THESE STEPS:

1-OPEN TERMINAL AND NAVIGATE TO THE PROJECT DIRECTORY.

2-COMPILE THE LEX FILE USING FLEX:
FLEX LEX.L

3-COMPILE THE GENERATED C FILE:

GCC LEX.YY.C -O LEXER

4-RUN THE LEXER WITH THE TEST FILES PROVIDED:

BASH

COPY CODE

```
./LEXER < TEST1.PAS
```

```
./LEXER < TEST2.PAS
```

```
./LEXER < TEST3.PAS
```

```
./LEXER < TEST4.PAS
```

- THIS WILL SHOW THE RECOGNIZED TOKENS AND PRINT THE SYMBOL TABLE AT THE END OF EACH TEST CASE.

TEST CASE:

1:-
./LEXER < TEST1.PAS

```
test1.pas
1  program demo;
2  var x : integer;
3

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(base) mac@Macs-MacBook-Pro Compiler -project1 % flex Lex.l
(base) mac@Macs-MacBook-Pro Compiler -project1 % gcc lex.yy.c -o lexer
(base) mac@Macs-MacBook-Pro Compiler -project1 % ./lexer < test1.pas

KEYWORD: program
ID: demo
SYMBOL: ;
KEYWORD: var
ID: x
SYMBOL: :
ID: integer
SYMBOL: ;

--- Symbol Table ---
program
demo
var
x
integer
(base) mac@Macs-MacBook-Pro Compiler -project1 % ./lexer < test2.pas
```

```
test2.pas
1  var a : integer;
2  |   b : string;
3  |   c : boolean;
4
5  |

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(base) mac@Macs-MacBook-Pro Compiler -project1 % ./lexer < test2.pas

KEYWORD: var
ID: a
SYMBOL: :
ID: integer
SYMBOL: ;
ID: b
SYMBOL: :
ID: string
SYMBOL: ;
ID: c
SYMBOL: :
ID: boolean
SYMBOL: ;

--- Symbol Table ---
var
a
integer
b
string
c
boolean
(base) mac@Macs-MacBook-Pro Compiler -project1 % ./lexer < test3.pas
```

2:-
./LEXER < TEST2.PAS

TEST CASE:

3:-

./LEXER < TEST3.PAS

```
test3.pas
1  begin
2      x := 5;
3      y := "Ali";
4      z := 10 + 2;
5  end.
6  |

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(base) mac@Macs-MacBook-Pro Compiler -project1 % ./lexer < test3.pas

KEYWORD: begin
ID: x
SYMBOL: :=
INT: 5
SYMBOL: ;
ID: y
SYMBOL: :=
STRING: "Ali"
SYMBOL: ;
ID: z
SYMBOL: :=
INT: 10
SYMBOL: +
INT: 2
SYMBOL: ;
KEYWORD: end
SYMBOL: .

--- Symbol Table ---
begin
x
y
z
end
(base) mac@Macs-MacBook-Pro Compiler -project1 % ./lexer < test4.pas
```

```
test4.pas
1  { This is a comment }
2  var name : string;
3  |

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(base) mac@Macs-MacBook-Pro Compiler -project1 % ./lexer < test4.pas

KEYWORD: var
ID: name
SYMBOL: :
ID: string
SYMBOL: ;

--- Symbol Table ---
var
name
string
(base) mac@Macs-MacBook-Pro Compiler -project1 %
```

4:-

./LEXER < TEST4.PAS

3- PART 2: SYNTAX ANALYSIS

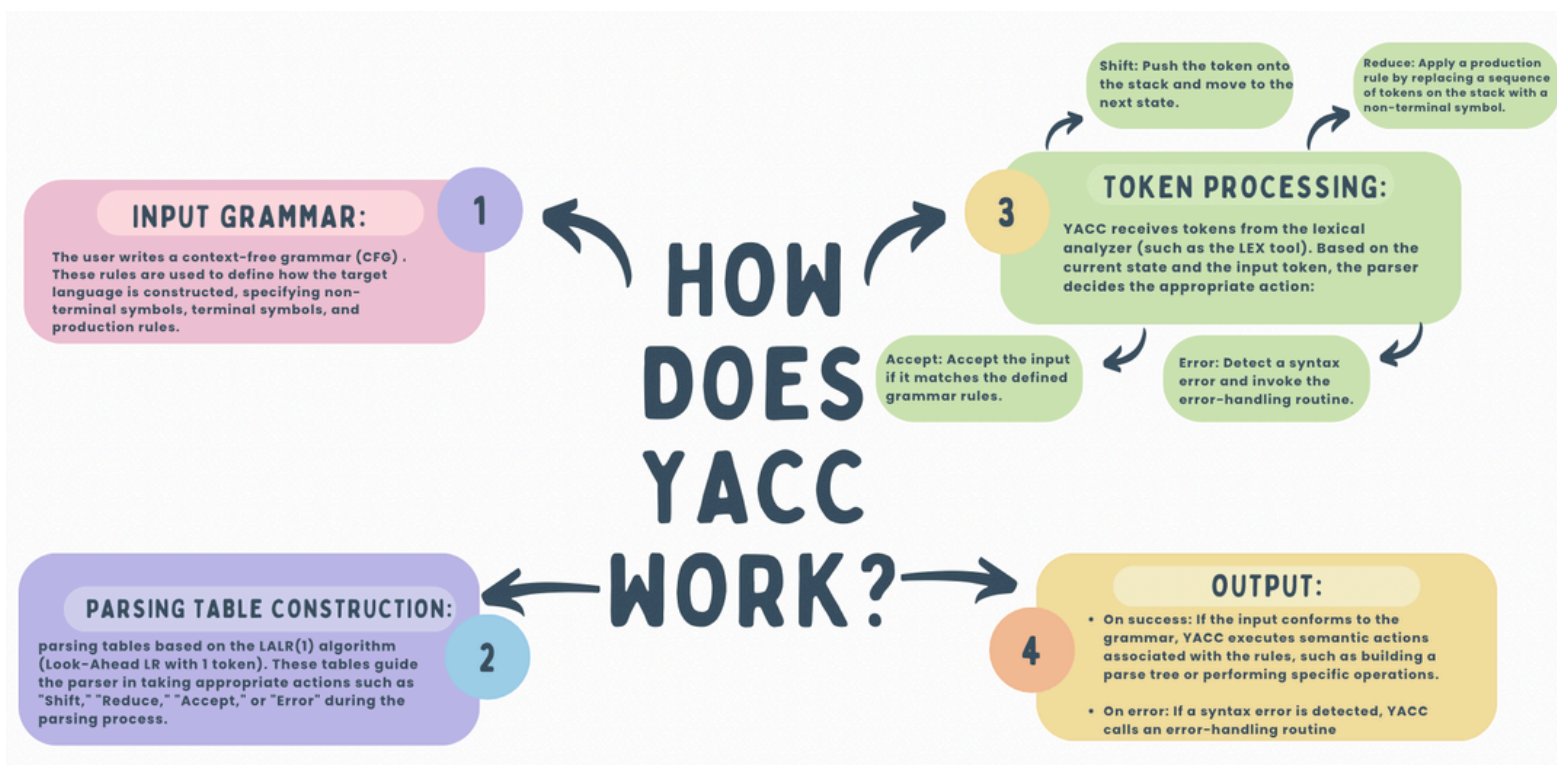
TASK 1: RESEARCH YACC AND UNDERSTAND ITS INTEGRATION WITH LEX AND ANSWER THE FOLLOWING QUESTIONS.

Q1: What is YACC?

YACC stands for Yet Another Compiler Compiler, a tool used to design compilers by automatically generating parsers based on specific grammar rules.

It is commonly used with another tool called Lex, which performs lexical analysis and converts the input text into tokens used by YACC.

Q2: How does YACC work?



3- PART 2: SYNTAX ANALYSIS

Q3: How does it integrate with LEX?

YACC integrates with LEX by receiving tokens returned from the Lexical Analyzer. Each token returned by LEX is matched with grammar rules in YACC to form a syntax tree. This allows building logic for program structure and semantics.

Q4: What is the structure of a YACC program?

A YACC program generally consists of three parts:

- Definition section: where tokens and types are declared
 - Rules section: grammar rules and associated actions
 - Code section: additional C code like main()
-

PARSING CODE:

```
E parser.y
1  %{
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  #define MAX_IDS 100
7  char* id_list[MAX_IDS];
8  int id_count = 0;
9
10 void print_production(const char *production_name) {
11     printf("%s", production_name);
12 }
13
14 extern void insert_symbol(const char* lexeme, const char* type);
15 extern void print_symbol_table();
16
17 void yyerror(const char *s);
18 int yylex(void);
19 %}
20
21 %union {
22     int ival;
23     char* sval;
24 }
25 %nonassoc LOWER_THAN_ELSE
26 %nonassoc ELSE
27
28 %left OR
29 %left AND
30 %left PLUS MINUS
31 %left MULT DIV MOD
32 %right NOT
33 %token <sval> ID STRING
34 %token <ival> INT
35
36 %token PROGRAM VAR BEGIN_KEYWORD END IF THEN ELSE WHILE DO FOR TO
37 %token PROCEDURE FUNCTION FORWARD TYPE ARRAY RECORD
38 %token OF NOT DIV MOD AND OR
39 %token PLUS MINUS MULT EQUAL LESS LESSEQUAL GREATER GREATEREQUAL NOTEQUAL DOT COMMA COLON SEMICOLON DOTDOT LPAREN RPAREN LBRACKET RBRACKET AS
40
41 %start Program
42
43 %%
```

```
E parser.y
45 Program:
46     PROGRAM ID SEMICOLON TypeDefinitions VariableDeclarations SubprogramDeclarations CompoundStatement DOT
47     {
48         insert_symbol($2, "program");
49         print_production("Program\n");
50     }
51     ;
52
53 TypeDefinitions:
54     /* empty */
55     | TYPE TypeDefinition SEMICOLON
56     { print_production("Type Definitions\n"); }
57     | TypeDefinitions TypeDefinition SEMICOLON
58     ;
59
60 TypeDefinition:
61     ID ASSIGN Type
62     { insert_symbol($1, "type"); print_production("Type definition\n"); }
63     ;
64
65 VariableDeclarations:
66     /* empty */
67     | VAR VariableDeclaration SEMICOLON
68     { print_production("Variable Declarations\n"); }
69     | VariableDeclarations VariableDeclaration SEMICOLON
70     ;
71
72 VariableDeclaration:
73     IdentifierList COLON Type
74     {
75         for (int i = 0; i < id_count; i++) {
76             insert_symbol(id_list[i], "variable");
77             free(id_list[i]);
78         }
79         id_count = 0;
80         print_production("Variable Declaration\n");
81     }
82     ;
83
84 IdentifierList:
85     ID
86     {
```

PARSING CODE:

```
E parser.y
84 IdentifierList:
85 {
86     if (id_count < MAX_IDS)
87     {
88         IdentifierList COMMA ID
89     }
90     {
91         if (id_count < MAX_IDS)
92             id_list[id_count++] = strdup($3);
93     }
94 }
95 ;
96
97 Type:
98 ID
99 | ARRAY LBACKET Constant DOTDOT Constant RBACKET OF Type
100 { print_production("Array type\n"); }
101 | RECORD FieldList END
102 { print_production("Record type\n"); }
103 ;
104
105 FieldList:
106 /* empty */
107 | IdentifierList COLON Type
108 | FieldList SEMICOLON IdentifierList COLON Type
109 ;
110
111 Constant:
112 INT
113 | MINUS INT
114 ;
115
116 SubprogramDeclarations:
117 /* empty */
118 | SubprogramDeclaration SEMICOLON
119 | SubprogramDeclarations SubprogramDeclaration SEMICOLON
120 ;
121
122 SubprogramDeclaration:
123 ProcedureDeclaration
124 | FunctionDeclaration
125 ;
126
127 ProcedureDeclaration:
128 PROCEDURE ID LPAREN FormalParameterList RPAREN SEMICOLON ProcedureBody
```

```
E parser.y
127 ProcedureDeclaration:
128 {
129     insert_symbol($2, "procedure");
130     print_production("Procedure declaration\n");
131 }
132 ;
133
134
135 ProcedureBody:
136 Block
137 | FORWARD
138 ;
139
140 FunctionDeclaration:
141 FUNCTION ID LPAREN FormalParameterList RPAREN COLON ResultType SEMICOLON FunctionBody
142 {
143     insert_symbol($2, "function");
144     print_production("Function declaration\n");
145 }
146 ;
147
148 FunctionBody:
149 Block
150 | FORWARD
151 ;
152
153 ResultType:
154 ID
155 ;
156
157 FormalParameterList:
158 /* empty */
159 | IdentifierList COLON Type
160 {
161     for (int i = 0; i < id_count; i++) {
162         insert_symbol(id_list[i], "parameter");
163         free(id_list[i]);
164     }
165     id_count = 0;
166 }
167 | FormalParameterList SEMICOLON IdentifierList COLON Type
168 {
169     for (int i = 0; i < id_count; i++) {
170         insert_symbol(id_list[i], "parameter");
```

PARSING CODE:

```

157 FormalParameterList:
158 {
159     for (int i = 0; i < id_count; i++) {
160         insert_symbol(id_list[i], "parameter");
161         free(id_list[i]);
162     }
163     id_count = 0;
164 }
165 ;
166
167 Block:
168     VariableDeclarations CompoundStatement
169     { print_production("Block\n"); }
170 ;
171
172 CompoundStatement:
173     BEGIN_KEYWORD StatementSequence END
174     { print_production("Compound statement\n"); }
175 ;
176
177 StatementSequence:
178     Statement
179     | StatementSequence SEMICOLON Statement
180 ;
181
182 Statement:
183     SimpleStatement
184     | StructuredStatement
185 ;
186
187 SimpleStatement:
188     /* empty */
189     | AssignmentStatement
190     | ProcedureStatement
191 ;
192
193 AssignmentStatement:
194     Variable ASSIGN Expression
195     { print_production("Assignment statement\n"); }
196 ;
197
198 ProcedureStatement:
199     ID LPAREN ActualParameterList RPAREN
200     { print_production("Procedure call\n"); }
201 ;

```

```

208 ProcedureStatement:
209     ID LPAREN ActualParameterList RPAREN
210     { print_production("Procedure call\n"); }
211 ;
212
213 StructuredStatement:
214     CompoundStatement
215     | IfStatement
216     | WhileStatement
217     | ForStatement
218 ;
219
220 IfStatement:
221     IF Expression THEN Statement %prec LOWER_THAN_ELSE
222     | IF Expression THEN Statement ELSE Statement
223     { print_production("If statement\n"); }
224 ;
225
226 WhileStatement:
227     WHILE Expression DO Statement
228     { print_production("While statement\n"); }
229 ;
230
231 ForStatement:
232     FOR ID ASSIGN Expression TO Expression DO Statement
233     { print_production("For statement\n"); }
234 ;
235
236 Expression:
237     SimpleExpression
238     | SimpleExpression RelationalOp SimpleExpression
239     { print_production("Expression with relation\n"); }
240 ;
241
242 RelationalOp:
243     LESS
244     | LESSEQUAL
245     | GREATER
246     | GREATEREQUAL
247     | NOTEQUAL
248     | EQUAL
249 ;
250
251 SimpleExpression:

```

PARSING CODE:

```

251 SimpleExpression:
252     | SimpleExpression AddOp Term
253     { print_production("Simple expression\n"); }
254     ;
255
256 AddOp:
257     PLUS
258     | MINUS
259     | OR
260     ;
261
262 Term:
263     Factor
264     | Term MulOp Factor
265     { print_production("Term with operation \n"); }
266     ;
267
268 MulOp:
269     MULT
270     | DIV
271     | MOD
272     | AND
273     ;
274
275 Factor:
276     INT
277     | STRING
278     | Variable
279     | FunctionReference
280     | NOT Factor
281     | LPAREN Expression RPAREN
282     ;
283
284 FunctionReference:
285     ID LPAREN ActualParameterList RPAREN
286     { print_production("Function reference \n"); }
287     ;
288
289 Variable:
290     ID ComponentSelection
291     ;
292
293 ComponentSelection:
294
```

```

295 Factor:
296     | NOT Factor
297     | LPAREN Expression RPAREN
298     ;
299
300 FunctionReference:
301     ID LPAREN ActualParameterList RPAREN
302     { print_production("Function reference \n"); }
303     ;
304
305 Variable:
306     ID ComponentSelection
307     ;
308
309 ComponentSelection:
310     /* empty */
311     | DOT ID
312     | LBRACKET Expression RBRACKET
313     ;
314
315 ActualParameterList:
316     /* empty */
317     | Expression
318     | Expression COMMA ActualParameterList
319     ;
320
321 %%
322
323 void yyerror(const char *s) {
324     fprintf(stderr, "Error: %s\n", s);
325 }
326
327 int main() {
328     printf("Starting YACC Parser...\n");
329     yyparse();
330     print_symbol_table();
331     return 0;
332 }
```

CODE EXPLANATION:

THIS YACC (BISON) PARSER, WRITTEN IN C, ANALYZES A PASCAL-LIKE PROGRAMMING LANGUAGE, BEGINNING WITH DECLARATIONS FOR STANDARD LIBRARIES, FUNCTION PROTOTYPES LIKE YYERROR (ERROR REPORTING), YYLEX (LEXICAL ANALYSIS), INSERT_SYMBOL (SYMBOL TABLE MANAGEMENT), AND PRINT_PRODUCTION (DEBUGGING OUTPUTS). THE %TOKEN DIRECTIVES DEFINE LEXICAL TOKENS (PROGRAM, ID, IF, ETC.), WHILE %UNION SPECIFIES TOKEN VALUE TYPES. GRAMMAR RULES ENFORCE THE LANGUAGE STRUCTURE, INCLUDING PROGRAM ENTRY (PROGRAM), TYPE/VARIABLE DECLARATIONS, EXPRESSIONS, PROCEDURES/FUNCTIONS, AND CONTROL STATEMENTS (IF, WHILE, FOR), WITH SEMANTIC ACTIONS ENABLING SYMBOL TABLE UPDATES OR LOGGING. THE YYERROR FUNCTION HANDLES SYNTAX ERRORS, AND MAIN INITIATES PARSING VIA YYPARSE(), COLLECTIVELY BUILDING A PARSER THAT VALIDATES AND PROCESSES PASCAL-LIKE SOURCE CODE.

HOW TO RUN THE CODE?!

- I INSTALLED DOCKER,
 - THEN ASKED CHATGPT TO GIVE ME A SET OF COMMANDS TO FOLLOW.
 - THE FIRST STEP WAS TO RUN DOCKER AND KEEP IT IN THE BACKGROUND.
 - THEN I NAVIGATED TO THE FOLDER WHERE I SAVED MY CODE AND THE TEST CASE.
 - I ADDED AN EXTRA SETUP FILE CALLED DOCKERFILE AND WROTE INSIDE IT A SET OF COMMANDS PROVIDED BY CHATGPT.
 - AFTER THAT, I OPENED THE TERMINAL AND STARTED ENTERING THE FOLLOWING COMMANDS ONE BY ONE:
 - **DOCKER BUILD -T LEX_IMAGE .**
 - **DOCKER RUN -IT --RM -V \${PWD}:/APP LEX_IMAGE BASH**
 - **APT-GET UPDATE**
 - **APT-GET INSTALL -Y FLEX BISON GCC**
 - **FLEX LEX.L**
 - **BISON -D PARSE.Y**
 - **./PARSER < TESTCASE1**
 - **./PARSER < TESTCASE2**
 - **./PARSER < TESTCASE3**
 - **./PARSER < TESTCASE4**
-

TEST CASE AND THE OUTPUT:

1:-

./PARSER < TESTCASE1

```
≡ TestCase1
1
2   program test1;
3   begin
4   | x := 5;
5   end.
```

```
root@3b540e9cc133:/app# ./parser < TestCase1
Starting YACC Parser...
Assignment statement
Compound statement
Program

Name          Type
-----
test1         program
root@3b540e9cc133:/app# ./parser < TestCase2
```

TEST CASE AND THE OUTPUT:

2:-

`./PARSER < TESTCASE2`

```
Testcase2
1
2   program test2;
3   var x, y: integer;
4   function add(a, b: integer): integer; forward;
5   begin
6   end.
7   |
```

```
root@3b540e9cc133:/app# ./parser < Testcase2
```

```
Starting YACC Parser...
```

```
Variable Declaration
```

```
Variable Declarations
```

```
Function declaration
```

```
Compound statement
```

```
Program
```

Name	Type
x	variable
y	variable
a	parameter
b	parameter
add	function
test2	program

TEST CASE AND THE OUTPUT:

3:-

./PARSER < TESTCASE3

```
TestCase3
1
2  program test3;
3  procedure printSum(a, b: integer);
4  begin
5  | result := a + b;
6  end;
7  begin
8  | printSum(5, 6);
9  end.
10
```

```
root@3b540e9cc133:/app# ./parser < TestCase3
Starting YACC Parser...
Simple expression
Assignment statement
Compound statement
Block
Procedure declaration
Procedure call
Compound statement
Program

Name                Type
-----
a                    parameter
b                    parameter
printSum             procedure
test3                program
```

TEST CASE AND THE OUTPUT:

4:-

./PARSER < TESTCASE4

```
testCase4
1
2  program test4;
3  function square(n: integer): integer;
4  begin
5    | square := n * n;
6  end;
7  begin
8    | result := square(4);
9  end.
```

```
root@3b540e9cc133:/app# ./parser < TestCase4
Starting YACC Parser...
Term with operation
Assignment statement
Compound statement
Block
Function declaration
Function reference
Assignment statement
Compound statement
Program

Name          Type
-----
n              parameter
square        function
test4         program
root@3b540e9cc133:/app#
```

4- PART 3: FINAL SUBMISSION COMPONENTS

- lexer.l: Complete LEX file including reserved symbol definitions and symbol table logic
 - parser.y: YACC file defining grammar rules and parsing actions
 - test.txt: Sample file with input code for testing
 - README.txt: Instructions to compile and run the project
-

5- CHALLENGES:

CHALLENGES WE FACED:

- FACED DIFFICULTY INTEGRATING LEX AND YACC, ESPECIALLY IN PASSING TOKENS AND MATCHING GRAMMAR RULES.
 - ENCOUNTERED ISSUES WITH WHITESPACE DEFINITIONS, WHICH CAUSED INCORRECT OUTPUT.
 - APPLYING THEORETICAL CONCEPTS IN PRACTICE WAS CHALLENGING DUE TO LIMITED CLEAR AND DETAILED RESOURCES.
 - THE OVERALL PROCESS OF LINKING COMPONENTS AND COMPLETING THE PROJECT REQUIRED SIGNIFICANT TIME AND EFFORT TO DEBUG AND EXECUTE PROPERLY.
-

6- SUMMARY

This project helped us understand how compilers begin by analyzing input text and converting it into structured data through tokens and grammar rules. It also demonstrated how tools like LEX and YACC simplify this process. We faced challenges in tool installation, debugging token rules, and connecting LEX and YACC smoothly, but the final integration worked correctly.

7- REFERENCES:

- [HTTPS://WWW.GNU.ORG/SOFTWARE/FLEX/](https://www.gnu.org/software/flex/)
 - [HTTPS://WWW.GNU.ORG/SOFTWARE/BISON/](https://www.gnu.org/software/bison/)
 - [HTTPS://WWW.GEEKSFORGEEKS.ORG/COMPILER-DESIGN/](https://www.geeksforgeeks.org/compiler-design/)
 - CHATGPT (USED FOR EXPLANATIONS AND SYNTAX DEBUGGING)
-

8-TASKS TABLE:

prepared the report, answered the questions, and explained the answers related to Lex and YACC,Writing a part of the lex rules.	تهاني البارقي
LEX code was implemented and tested,and the functionality of the code was clearly explained. REVIEWING ALL THE CODE TO ENSURE CORRECT OUTPUT	نهلة البقمي
The YACC code was implemented and the functionality of the code was clearly explained, And wrote an explanation of the codes.	سوسن عمر
Writing and executing the Lex code, answering some questions related to the Lex and parsing code, integrating both codes, and making the necessary modifications.	ضحى النباتي
Writing part of the Lex rules and answering some questions.	عائشه العدواني